HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Môn học: Phân tích và thiết kế hệ thống thông tin

TIỂU LUẬN MÔN HỌC

CHAPTER 4. DESIGN

Từ yêu cầu đến lập trình – Phân tích và thiết kế ứng dụng thương mại điện tử có tìm kiếm đa phương thức

Sinh viên: Vũ Hùng Anh

Mã sinh viên: B23DCDT022

Lớp: 04

Hà Nội, ngày 28 tháng 10 năm 2026

Mục lục

Chapter 4. Design – Thiết kế hệ thống **2**

4.1 Class Diagram in Analysis – Sơ đồ lớp phân tích **2**
4.1.1 Update – Cập nhật sơ đồ so với Bài kiểm tra 01 . . . 2
4.1.2 Describe in words – Mô tả sơ đồ bằng lời . . . 3
4.1.3 Explain relationships – Giải thích các quan hệ . . . 6

4.2 Data Model – Database **7**
4.2.1 Data model – Mô hình dữ liệu ORM . . . 7
4.2.2 From data model to database – Quy tắc ánh xạ lớp sang bảng . . . 9
4.2.3 Explain tables – Giải thích các bảng . . . 9
4.2.4 Explain relationships – Giải thích quan hệ giữa các bảng . . . 11
4.2.5 Generate DB – Sinh cơ sở dữ liệu . . . 13

4.3 Class Diagram in Design – Sơ đồ lớp thiết kế **14**
4.3.1 Update attributes – Cập nhật thuộc tính . . . 16
4.3.2 Update methods – Cập nhật phương thức . . . 16
4.3.3 Update relationships – Cập nhật và giải thích quan hệ . . . 18

4.4 Layer + Class + Relationship – Kiến trúc phân tầng **20**
4.4.1 Layers – Bốn tầng và quy tắc phụ thuộc . . . 20
4.4.2 Classes and relationships – Lớp và quan hệ trong từng tầng . . . 21
4.4.3 Explain – Checkout đi qua các tầng . . . 23
4.4.4 Explain – Gói search . . . 24

4.5 Generate code from 4.4 – Sinh mã nguồn **25**
4.5.1 Generate code – Cách sinh mã từ mô hình 4.4 . . . 25
4.5.2 Explain packages – Giải thích các package . . . 25
4.5.3 Class code – Giải thích mã của các lớp . . . 27
4.5.4 Biên dịch và kiểm tra nhanh . . . 30
4.5.5 Giới hạn của mã sinh ra . . . 30

4.6 Conclusion – Kết luận chương **31**

Phụ lục – Tệp nộp kèm và cách tái tạo **32**

CHAPTER 4. DESIGN – THIẾT KẾ HỆ THỐNG

Chương này được viết lại theo dàn ý 4.1–4.6 mà giảng viên giao cho phần Design, thay cho chương Design của bản tiểu luận V1. Bản V1 trình bày nguyên tắc thiết kế bằng lời và chưa có sơ đồ thiết kế; bản này đi qua một chuỗi sản phẩm cụ thể: sơ đồ lớp phân tích được cập nhật, mô hình dữ liệu và cơ sở dữ liệu sinh từ nó, sơ đồ lớp thiết kế, kiến trúc phân tầng, và mã nguồn Java sinh từ mô hình phân tầng.

Nguồn đầu vào gồm: mô hình 19 lớp của Bài kiểm tra 01 (ba project Visual Paradigm `ecomAnalysis`, `eComDataModel`, `eComDesign`); slide A03 *Domain Model and Analysis Class Diagram* (Cart, CartItem, Order, OrderItem và phép biến đổi checkout); slide A05 *From Analysis to Design* (Controller – Service – Domain – Repository, layer và package); và prototype tìm kiếm đa phương thức Assignment 06. Toàn bộ sơ đồ được dựng trong Visual Paradigm 18.1 Community Edition qua Open API thành một project chỉnh sửa được; hình trong bài là ảnh xuất trực tiếp từ project đó. Lớp, thuộc tính, phương thức, bảng và khóa đều lấy từ một tệp đặc tả chung (`tools/model.py`), nên sơ đồ, câu lệnh SQL và mã Java dùng đúng cùng một tên gọi.

Bảng 4.1: Luồng sản phẩm của chương Design

Mục	Câu hỏi trả lời	Đầu vào	Sản phẩm
4.1	Hệ thống cần những khái niệm nào?	19 lớp Bài kiểm tra 01, slide A03	Sơ đồ lớp phân tích 20 lớp (Hình 4.1)
4.2	Dữ liệu được lưu như thế nào?	Sơ đồ 4.1	Data model ORM (Hình 4.2), mô hình CSDL 20 bảng (Hình 4.3), tệp `db/ecom_schema.sql`
4.3	Mỗi lớp giữ dữ liệu và hành vi gì?	Sơ đồ 4.1, slide A05	Sơ đồ lớp thiết kế có kiểu, phương thức, điều hướng (Hình 4.4)
4.4	Phần mềm chia thành tầng nào, lớp nào gọi lớp nào?	Sơ đồ 4.3, prototype A6	Sơ đồ layer (Hình 4.5), sơ đồ lớp theo tầng (Hình 4.6), sequence checkout (Hình 4.7)
4.5	Mã nguồn tương ứng ra sao?	Mô hình 4.4	Mã Java trong `code/src`, chia theo package của 4.4

4.1 Class Diagram in Analysis – Sơ đồ lớp phân tích

Sơ đồ lớp phân tích trả lời câu hỏi *hệ thống cần biểu diễn những khái niệm nào và chúng liên hệ với nhau ra sao*. Theo slide A03, ở mức này chưa đưa vào Controller, Service, Repository hay bảng dữ liệu; mỗi lớp là một khái niệm nghiệp vụ, mỗi quan hệ là một ràng buộc nghiệp vụ có bội số.

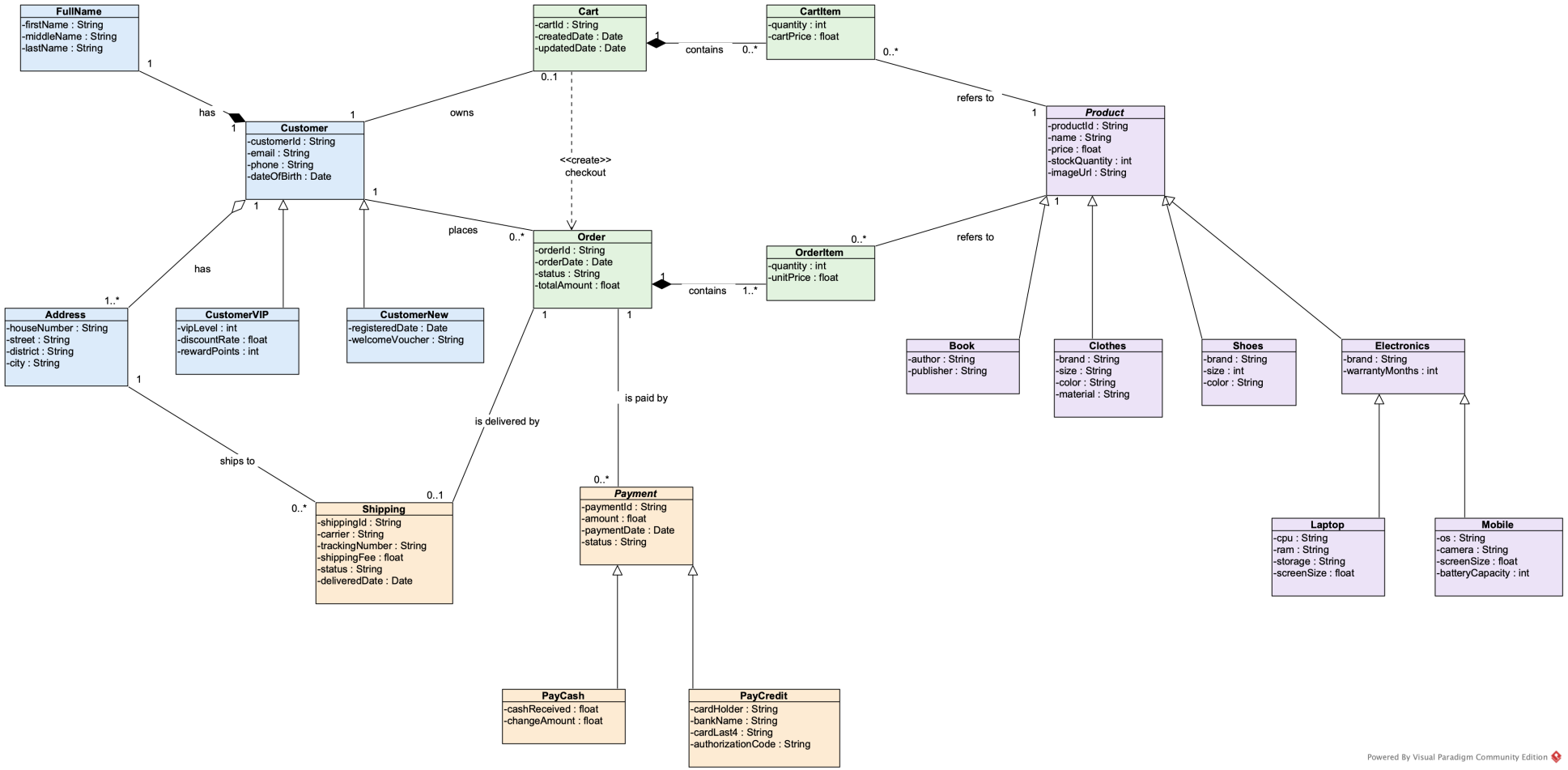
4.1.1 Update – Cập nhật sơ đồ so với Bài kiểm tra 01

Sơ đồ của Bài kiểm tra 01 dùng đúng 19 lớp do đề cho. Khi dùng nó làm đầu vào cho thiết kế, có một số điểm làm phần sau phức tạp không cần thiết, nên sơ đồ được cập nhật như Bảng 4.2. Kết quả có 20 lớp: 19 lớp cũ và lớp trừu tượng `Product`.

Bảng 4.2: Các thay đổi của sơ đồ phân tích so với Bài kiểm tra 01

Điểm thay đổi	Bài kiểm tra 01	Bản cập nhật	Lý do
Lớp cha của hàng hóa	Không có `Product`; `CartItem` và `OrderItem` mỗi lớp có bốn liên kết tới `Book`, `Clothes`, `Shoes`, `Electronics` với ràng buộc {xor}	Thêm lớp trừu tượng `Product`, là lớp cha của `Book`, `Clothes`, `Shoes`, `Electronics`. `CartItem` và `OrderItem` chỉ còn một liên kết *refers to* tới `Product`, bội số 0..* – 1	Một dòng hàng luôn chỉ tới đúng một sản phẩm mà không cần ràng buộc {xor}; khớp mô hình Cart → CartItem → Product của slide A03; khi lưu trữ chỉ cần một khóa ngoại `product_id`
Thuộc tính chung của hàng hóa	`productId`, `price`, `stockQuantity` và tên hàng lặp lại ở bốn lớp; `Book` dùng `title`	Chuyển `productId`, `name`, `price`, `stockQuantity` lên `Product`; `Book.title` thành `Product.name`; thêm `Product.imageUrl`	Bỏ trùng lặp; `imageUrl` cần cho chức năng tìm kiếm bằng ảnh ở Chương 3
Thẻ thanh toán	`PayCredit` có `cardNumber`, `cardHolder`, `bankName`, `expiryDate`	`PayCredit` có `cardHolder`, `bankName`, `cardLast4`, `authorizationCode`	Theo quyết định D-01 (Chương 1), hệ thống không lưu số thẻ đầy đủ; chỉ giữ bốn số cuối và mã cấp phép do cổng thanh toán trả về
Bội số và tên quan hệ	Chỉ bổ sung ở data model (Câu 2)	Ghi ngay trên sơ đồ phân tích; `Order` – `Payment` là 1 – 0..*	Slide A03 yêu cầu ghi bội số ở mức phân tích; một đơn có thể có nhiều lần thanh toán khi lần trước thất bại
Phụ thuộc checkout	`Cart` ⇢ `Order` «create» *checkout*	Giữ nguyên	Slide A03: checkout là một phép biến đổi Cart thành Order, không phải liên kết cố định

4.1.2 Describe in words – Mô tả sơ đồ bằng lời



Hình 4.1: Sơ đồ lớp phân tích đã cập nhật (20 lớp, Visual Paradigm). Lớp in nghiêng là lớp trừu tượng.

Hình 4.1 gồm bốn nhóm lớp.

Nhóm khách hàng. `Customer` là người mua, định danh bằng `customerId` và có `email`, `phone`, `dateOfBirth`. Họ tên được tách thành đối tượng giá trị `FullName` (`firstName`, `middleName`, `lastName`). Mỗi khách có một hoặc nhiều `Address` (`houseNumber`, `street`, `district`, `city`). Hai hạng khách là `CustomerVIP` (`vipLevel`, `discountRate`, `rewardPoints`) và `CustomerNew` (`registeredDate`, `welcomeVoucher`).

Nhóm sản phẩm. `Product` là lớp trừu tượng chứa phần chung của mọi mặt hàng: `productId`, `name`, `price`, `stockQuantity`, `imageUrl`. Bốn nhóm hàng chuyên biệt hóa `Product`: `Book` (`author`, `publisher`), `Clothes` (`brand`, `size`, `color`, `material`), `Shoes` (`brand`, `size`, `color`) và `Electronics` (`brand`, `warrantyMonths`). `Laptop` (`cpu`, `ram`, `storage`, `screenSize`) và `Mobile` (`os`, `camera`, `screenSize`, `batteryCapacity`) chuyên biệt hóa tiếp `Electronics`.

Nhóm giỏ hàng và đơn hàng. `Cart` (`cartId`, `createdDate`, `updatedDate`) là trạng thái mua sắm tạm thời, gồm các `CartItem` (`quantity`, `cartPrice`). `Order` (`orderId`, `orderDate`, `status`, `totalAmount`) là giao dịch đã xác nhận, gồm các `OrderItem` (`quantity`, `unitPrice`).

Nhóm giao hàng và thanh toán. `Shipping` (`shippingId`, `carrier`, `trackingNumber`, `shippingFee`, `status`, `deliveredDate`) ghi thông tin giao một đơn tới một địa chỉ. `Payment` là lớp trừu tượng (`paymentId`, `amount`, `paymentDate`, `status`) với hai hình thức `PayCash` (`cashReceived`, `changeAmount`) và `PayCredit` (`cardHolder`, `bankName`, `cardLast4`, `authorizationCode`).

Bảng 4.3: Ý nghĩa của 20 lớp phân tích

Lớp	Ý nghĩa nghiệp vụ	Thuộc tính
`Customer`	Người mua có tài khoản	`customerId`, `email`, `phone`, `dateOfBirth`
`FullName`	Họ tên của khách, đối tượng giá trị	`firstName`, `middleName`, `lastName`
`Address`	Địa chỉ liên hệ và nhận hàng	`houseNumber`, `street`, `district`, `city`
`CustomerVIP`	Khách hạng VIP, được chiết khấu và tích điểm	`vipLevel`, `discountRate`, `rewardPoints`
`CustomerNew`	Khách mới đăng ký, có thể có voucher chào mừng	`registeredDate`, `welcomeVoucher`
`Product`	Mặt hàng được bán (trừu tượng)	`productId`, `name`, `price`, `stockQuantity`, `imageUrl`
`Book`	Sách	`author`, `publisher`
`Clothes`	Quần áo	`brand`, `size`, `color`, `material`
`Shoes`	Giày	`brand`, `size`, `color`
`Electronics`	Hàng điện tử	`brand`, `warrantyMonths`
`Laptop`	Máy tính xách tay	`cpu`, `ram`, `storage`, `screenSize`
`Mobile`	Điện thoại di động	`os`, `camera`, `screenSize`, `batteryCapacity`
`Cart`	Giỏ hàng đang mua dở	`cartId`, `createdDate`, `updatedDate`
`CartItem`	Một sản phẩm và số lượng định mua trong giỏ	`quantity`, `cartPrice` (giá lúc thêm vào giỏ)
`Order`	Đơn hàng đã xác nhận	`orderId`, `orderDate`, `status`, `totalAmount`
`OrderItem`	Một sản phẩm đã mua trong đơn	`quantity`, `unitPrice` (giá chốt lúc đặt)
`Shipping`	Thông tin giao hàng của một đơn	`shippingId`, `carrier`, `trackingNumber`, `shippingFee`, `status`, `deliveredDate`
`Payment`	Một lần thanh toán (trừu tượng)	`paymentId`, `amount`, `paymentDate`, `status`

Lớp	Ý nghĩa nghiệp vụ	Thuộc tính
`PayCash`	Thanh toán tiền mặt	`cashReceived`, `changeAmount`
`PayCredit`	Thanh toán thẻ qua cổng thanh toán	`cardHolder`, `bankName`, `cardLast4`, `authorizationCode`

4.1.3 Explain relationships – Giải thích các quan hệ

Bảng 4.4 giải thích từng quan hệ trên Hình 4.1. Bội số ghi ở một đầu cho biết *một* đối tượng ở đầu đối diện liên kết với bao nhiêu đối tượng ở đầu này.

Bảng 4.4: Quan hệ giữa các lớp phân tích

Quan hệ (tên)	Loại	Bội số	Ý nghĩa nghiệp vụ
`Customer` – `FullName` (has)	Composition	1 – 1	Mỗi khách có đúng một họ tên; họ tên không tồn tại ngoài khách hàng nên mất cùng khách.
`Customer` – `Address` (has)	Aggregation	1 – 1..*	Khách phải có ít nhất một địa chỉ để nhận hàng. Địa chỉ còn được `Shipping` tham chiếu nên không bị xóa theo khách, vì vậy dùng aggregation chứ không dùng composition.
`Customer` – `Cart` (owns)	Association	1 – 0..1	Mỗi khách có tối đa một giỏ đang hoạt động; khách mới có thể chưa có giỏ.
`Customer` – `Order` (places)	Association	1 – 0..*	Khách đặt nhiều đơn; khách mới chưa có đơn. Đơn tồn tại độc lập để còn đối soát.
`Cart` – `CartItem` (contains)	Composition	1 – 0..*	Dòng giỏ chỉ tồn tại trong giỏ; giỏ rỗng là trạng thái hợp lệ.
`Order` – `OrderItem` (contains)	Composition	1 – 1..*	Dòng đơn là bộ phận của đơn; đơn không có dòng hàng nào là vô nghĩa.
`CartItem` – `Product` (refers to)	Association	0..* – 1	Mỗi dòng giỏ chỉ tới đúng một sản phẩm; một sản phẩm có thể nằm trong nhiều giỏ.
`OrderItem` – `Product` (refers to)	Association	0..* – 1	Mỗi dòng đơn chỉ tới đúng một sản phẩm; một sản phẩm có thể có trong nhiều đơn.
`Order` – `Shipping` (is delivered by)	Association	1 – 0..1	Đơn mới tạo có thể chưa có thông tin giao; khi giao thì có đúng một bản ghi giao hàng.
`Shipping` – `Address` (ships to)	Association	0..* – 1	Mỗi lần giao tới đúng một địa chỉ; một địa chỉ nhận được nhiều lần giao.
`Order` – `Payment` (is paid by)	Association	1 – 0..*	Đơn có thể chưa thanh toán, hoặc có nhiều lần thanh toán khi lần trước thất bại.
`CustomerVIP`, `CustomerNew` → `Customer`	Generalization		Hai hạng khách dùng lại thuộc tính của `Customer` và thêm thuộc tính riêng.
`Book`, `Clothes`, `Shoes`, `Electronics` → `Product`	Generalization		Bốn nhóm hàng dùng chung mã, tên, giá, tồn kho, ảnh; mỗi nhóm thêm thuộc tính riêng.
`Laptop`, `Mobile` → `Electronics`	Generalization		Hai loại thiết bị kế thừa `brand`, `warrantyMonths` và phần chung của `Product`.
`PayCash`, `PayCredit` → `Payment`	Generalization		Hai hình thức thanh toán dùng chung số tiền, thời điểm, trạng thái.

Quan hệ (tên)	Loại	Bội số	Ý nghĩa nghiệp vụ
`Cart` `--→` `Order` (checkout)	Dependency «create»		Checkout dùng nội dung giỏ để tạo một đơn mới; sau đó giỏ được làm rỗng.

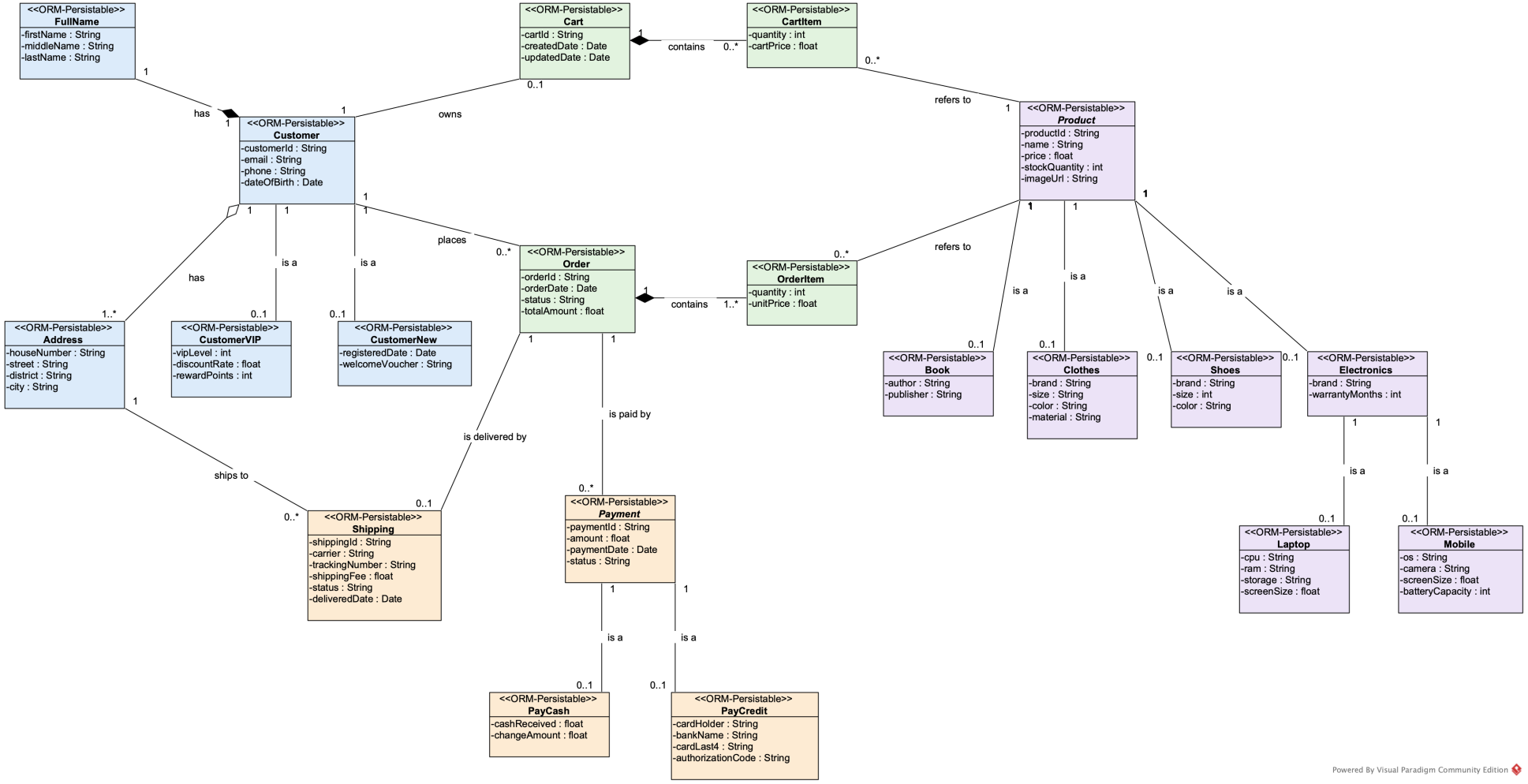
`Product` và `Payment` là lớp trừu tượng (tên in nghiêng) vì không có mặt hàng nào không thuộc một nhóm cụ thể và không có lần thanh toán nào không thuộc một hình thức cụ thể.

CartItem khác OrderItem, và checkout là một phép biến đổi. Hai lớp có thuộc tính giống nhau nhưng khác ý nghĩa (slide A03). `CartItem` là thông tin mua sắm tạm thời: `cartPrice` là giá lúc khách thêm hàng vào giỏ và có thể thay đổi. `OrderItem` là thông tin giao dịch lịch sử: `unitPrice` là giá được chốt khi tạo đơn và không đổi khi giá catalog thay đổi sau đó. Checkout biến giỏ $\{\text{CartItem}_1, \ldots, \text{CartItem}_n\}$ thành đơn $\{\text{OrderItem}_1, \ldots, \text{OrderItem}_n\}$ với cùng sản phẩm và số lượng, sau đó giỏ trở thành rỗng. Vì giỏ và đơn là hai trạng thái nghiệp vụ khác nhau, quan hệ giữa `Cart` và `Order` được vẽ là phụ thuộc «create» thay vì một association cố định.

4.2 Data Model – Database

Mục này chuyển sơ đồ phân tích thành mô hình dữ liệu (data model) rồi sinh cơ sở dữ liệu. Có ba sản phẩm: data model dạng lớp ORM trong Visual Paradigm (Hình 4.2), mô hình cơ sở dữ liệu vật lý gồm 20 bảng (Hình 4.3) và tệp DDL `db/ecom_schema.sql`.

4.2.1 Data model – Mô hình dữ liệu ORM



Hình 4.2: Data model: 20 lớp ORM-Persistable, quan hệ kế thừa đã chuyển thành association *is a* 1 – 0..1.

Data model trong Hình 4.2 được tạo từ Hình 4.1 theo cùng bốn bước như Câu 2 của Bài kiểm tra 01:

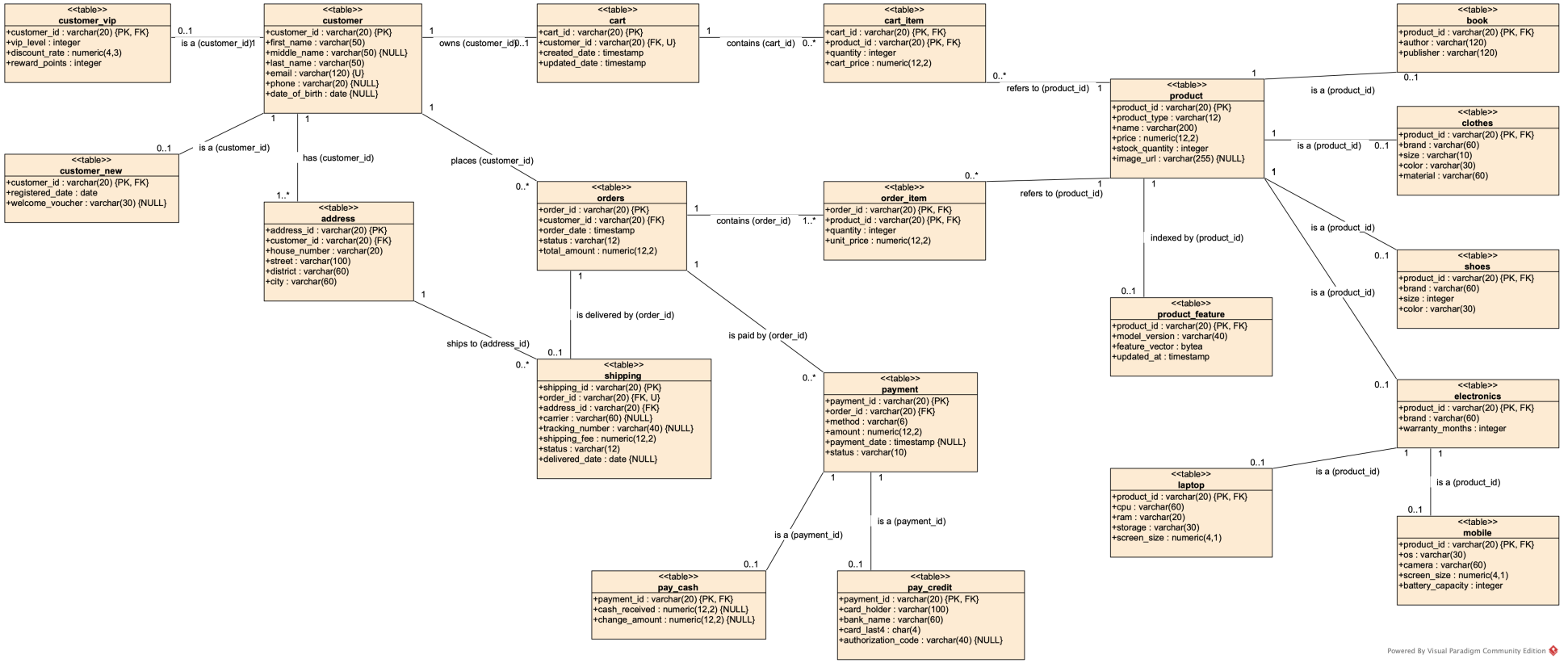
1. mọi lớp được gắn stereotype `<<ORM–Persistable>>`, nghĩa là đối tượng của lớp được lưu thành bản ghi;
2. mọi association có tên và bội số ở hai đầu;
3. mỗi quan hệ kế thừa được đổi thành association *is a*, đầu lớp cha có bội số 1, đầu lớp con có bội số 0..1, vì mô hình quan hệ không có khái niệm kế thừa;
4. phụ thuộc «create» *checkout* bị bỏ vì nó mô tả một thao tác, không phải dữ liệu cần lưu.

4.2.2 From data model to database – Quy tắc ánh xạ lớp sang bảng

Bảng 4.5: Quy tắc chuyển data model thành bảng

Trường hợp	Quy tắc	Ví dụ
Lớp thực thể	Mỗi lớp thành một bảng, tên viết thường nối bằng gạch dưới. `Order` đổi thành `orders` vì `ORDER` là từ khóa SQL.	`CartItem` → `cart_item`
Định danh	Thuộc tính định danh thành khóa chính kiểu `varchar(20)`.	`customer_id`, `product_id`, `order_id`
Đối tượng giá trị 1 – 1	Composition 1 – 1 với `FullName` được nhúng thành cột của bảng chủ, không tạo bảng riêng.	`customer.first_name`, `middle_name`, `last_name`
Kế thừa	Mỗi lớp một bảng (class table inheritance): bảng con dùng khóa chính đồng thời là khóa ngoại tới bảng cha.	`book.product_id` → `product`; `laptop.product_id` → `electronics`
Cột phân loại	Bảng cha có cột cho biết bản ghi thuộc lớp con nào.	`product.product_type`, `payment.method`
Association 1 – *	Khóa ngoại đặt ở bảng phía nhiều.	`orders.customer_id`, `payment.order_id`
Association 1 – 0..1	Khóa ngoại kèm ràng buộc `UNIQUE`.	`cart.customer_id`, `shipping.order_id`
Composition có dòng hàng	Bảng dòng hàng dùng khóa chính ghép; xóa bản ghi cha thì xóa luôn dòng hàng (`ON DELETE CASCADE`).	`cart_item(cart_id, product_id)`, `order_item(order_id, product_id)`
Tiền, số lượng, trạng thái	Tiền dùng `numeric(12,2)`; số lượng là `integer` có `CHECK`; trạng thái là chuỗi có `CHECK ... IN (...)`.	`order_item.unit_price`, `cart_item.quantity > 0`
Bảng kỹ thuật	Bảng `product_feature` lưu vector đặc trưng ảnh cho phân hệ tìm kiếm; không có trong mô hình phân tích, được thêm vì thiết kế gói search (mục 4.4.4).	`product_feature(product_id, model_version, feature_vector, updated_at)`

4.2.3 Explain tables – Giải thích các bảng



Hình 4.3: Mô hình cơ sở dữ liệu vật lý của eCom (Visual Paradigm): 20 bảng «table»; mỗi cột ghi kiểu SQL và {PK} khóa chính, {FK} khóa ngoại, {U} duy nhất, {NULL} cho phép rỗng; mỗi đường nối là một khóa ngoại, ghi tên quan hệ, cột khóa ngoại và bội số.

Hình 4.3 là mô hình cơ sở dữ liệu vật lý dựng trong Visual Paradigm theo các quy tắc ở Bảng 4.5. Mỗi bảng được vẽ như một lớp có stereotype «table»: tên cột, kiểu SQL và các khóa nằm ngay trong hộp, nên đọc được toàn bộ lược đồ trên một hình. Đường nối đi từ bảng cha tới bảng con, ghi tên quan hệ và cột khóa ngoại, ví dụ *contains (order_id)* từ `orders` tới `order_item`. Project VP còn có sơ đồ ERD (4.2c) dựng từ cùng đặc tả bằng phần tử bảng, cột và khóa ngoại của VP. Bảng 4.6 liệt kê 20 bảng cùng lớp nguồn của chúng.

Bảng 4.6: Danh sách bảng của cơ sở dữ liệu

#	Bảng	Khóa chính	Khóa ngoại → bảng cha	Lớp nguồn
1	`customer`	`customer_id`	–	`Customer` + `FullName`
2	`customer_vip`	`customer_id`	`customer_id` → `customer`	`CustomerVIP`
3	`customer_new`	`customer_id`	`customer_id` → `customer`	`CustomerNew`
4	`address`	`address_id`	`customer_id` → `customer`	`Address`
5	`product`	`product_id`	–	`Product`
6	`book`	`product_id`	`product_id` → `product`	`Book`
7	`clothes`	`product_id`	`product_id` → `product`	`Clothes`
8	`shoes`	`product_id`	`product_id` → `product`	`Shoes`
9	`electronics`	`product_id`	`product_id` → `product`	`Electronics`
10	`laptop`	`product_id`	`product_id` → `electronics`	`Laptop`
11	`mobile`	`product_id`	`product_id` → `electronics`	`Mobile`
12	`product_feature`	`product_id`	`product_id` → `product`	`VectorIndex` (gói search)
13	`cart`	`cart_id`	`customer_id` → `customer` (UNIQUE)	`Cart`
14	`cart_item`	`cart_id`, `product_id`	`cart_id` → `cart`; `product_id` → `product`	`CartItem`
15	`orders`	`order_id`	`customer_id` → `customer`	`Order`
16	`order_item`	`order_id`, `product_id`	`order_id` → `orders`; `product_id` → `product`	`OrderItem`
17	`shipping`	`shipping_id`	`order_id` → `orders` (UNIQUE); `address_id` → `address`	`Shipping`
18	`payment`	`payment_id`	`order_id` → `orders`	`Payment`
19	`pay_cash`	`payment_id`	`payment_id` → `payment`	`PayCash`
20	`pay_credit`	`payment_id`	`payment_id` → `payment`	`PayCredit`

4.2.4 Explain relationships – Giải thích quan hệ giữa các bảng

Bảng 4.7 giải thích 21 khóa ngoại. Cột bội số ghi theo thứ tự bảng cha – bảng con.

Bảng 4.7: Khóa ngoại, tên quan hệ và bội số

Khóa ngoại (bảng con.cột → bảng cha)	Tên quan hệ	Bội số	Ý nghĩa
`customer_vip.customer_id` → `customer`	is a	1 – 0..1	Phần riêng của khách VIP
`customer_new.customer_id` → `customer`	is a	1 – 0..1	Phần riêng của khách mới
`address.customer_id` → `customer`	has	1 – 1..*	Địa chỉ của khách
`book.product_id` → `product`	is a	1 – 0..1	Phần riêng của sách
`clothes.product_id` → `product`	is a	1 – 0..1	Phần riêng của quần áo
`shoes.product_id` → `product`	is a	1 – 0..1	Phần riêng của giày
`electronics.product_id` → `product`	is a	1 – 0..1	Phần riêng của hàng điện tử
`laptop.product_id` → `electronics`	is a	1 – 0..1	Phần riêng của laptop
`mobile.product_id` → `electronics`	is a	1 – 0..1	Phần riêng của điện thoại
`product_feature.product_id` → `product`	indexed by	1 – 0..1	Vector đặc trưng ảnh của sản phẩm
`cart.customer_id` → `customer`	owns	1 – 0..1	Mỗi khách tối đa một giỏ (UNIQUE)
`cart_item.cart_id` → `cart`	contains	1 – 0..*	Dòng của giỏ
`cart_item.product_id` → `product`	refers to	1 – 0..*	Sản phẩm của dòng giỏ
`orders.customer_id` → `customer`	places	1 – 0..*	Người đặt đơn
`order_item.order_id` → `orders`	contains	1 – 1..*	Dòng của đơn
`order_item.product_id` → `product`	refers to	1 – 0..*	Sản phẩm đã mua
`shipping.order_id` → `orders`	is delivered by	1 – 0..1	Mỗi đơn tối đa một bản ghi giao (UNIQUE)
`shipping.address_id` → `address`	ships to	1 – 0..*	Địa chỉ nhận hàng
`payment.order_id` → `orders`	is paid by	1 – 0..*	Các lần thanh toán của đơn
`pay_cash.payment_id` → `payment`	is a	1 – 0..1	Phần riêng của thanh toán tiền mặt
`pay_credit.payment_id` → `payment`	is a	1 – 0..1	Phần riêng của thanh toán thẻ

Khóa ngoại và `UNIQUE` bảo đảm cận trên của bội số, nhưng không tự bảo đảm cận dưới 1 ở các quan hệ 1 – 1..*. Điều kiện "mỗi đơn có ít nhất một dòng hàng" và "mỗi khách có ít nhất một địa chỉ" được kiểm tra ở tầng nghiệp vụ: `CheckoutService.checkout` từ chối giỏ rỗng (mục 4.4.3). Tương tự, cột

phân loại cho biết bản ghi cha phải có đúng một bản ghi con nào; điều này cũng do tầng nghiệp vụ giữ.

4.2.5 Generate DB – Sinh cơ sở dữ liệu

Tệp `db/ecom_schema.sql` (200 dòng, 20 bảng, cú pháp PostgreSQL) được sinh bằng `tools/gen_sql.py` từ cùng đặc tả bảng đã dùng để dựng Hình 4.3 trong Visual Paradigm. Bảng được tạo theo thứ tự phụ thuộc khóa ngoại: bảng cha trước, bảng con sau. Đoạn trích dưới đây gồm bảng `product` và hai cặp bảng dòng hàng, thể hiện các quy tắc khóa chính ghép, `CHECK` và `ON DELETE CASCADE`.

```sql
-- Product
CREATE TABLE product (
    product_id     varchar(20)   NOT NULL PRIMARY KEY,
    product_type   varchar(12)   NOT NULL CHECK (product_type IN ('BOOK','CLOTHES','SHOES','ELECTRONICS','LAPTOP','
     MOBILE')),
    name           varchar(200)  NOT NULL,
    price          numeric(12,2) NOT NULL CHECK (price >= 0),
    stock_quantity integer       NOT NULL CHECK (stock_quantity >= 0),
    image_url      varchar(255)
);

-- CartItem
CREATE TABLE cart_item (
    cart_id    varchar(20)   NOT NULL REFERENCES cart (cart_id) ON DELETE CASCADE,
    product_id varchar(20)   NOT NULL REFERENCES product (product_id),
    quantity   integer       NOT NULL CHECK (quantity > 0),
    cart_price numeric(12,2) NOT NULL CHECK (cart_price >= 0),
    PRIMARY KEY (cart_id, product_id)
);

-- Order
CREATE TABLE orders (
    order_id     varchar(20)   NOT NULL PRIMARY KEY,
    customer_id  varchar(20)   NOT NULL REFERENCES customer (customer_id),
    order_date   timestamp     NOT NULL,
    status       varchar(12)   NOT NULL CHECK (status IN ('PLACED','PROCESSING','SHIPPED','DELIVERED','CANCELLED')),
    total_amount numeric(12,2) NOT NULL CHECK (total_amount >= 0)
);

-- OrderItem
CREATE TABLE order_item (
    order_id   varchar(20)   NOT NULL REFERENCES orders (order_id) ON DELETE CASCADE,
    product_id varchar(20)   NOT NULL REFERENCES product (product_id),
    quantity   integer       NOT NULL CHECK (quantity > 0),
    unit_price numeric(12,2) NOT NULL CHECK (unit_price >= 0),
    PRIMARY KEY (order_id, product_id)
);
```

Tệp DDL được chạy thử trên SQLite trong bộ nhớ với ràng buộc khóa ngoại bật, nạp dữ liệu mẫu, rồi thử các câu lệnh vi phạm ràng buộc. Kết quả do script ghi lại như sau:

```
Validation of db/ecom_schema.sql with SQLite 3.53.4 (:memory:, PRAGMA foreign_keys=ON)

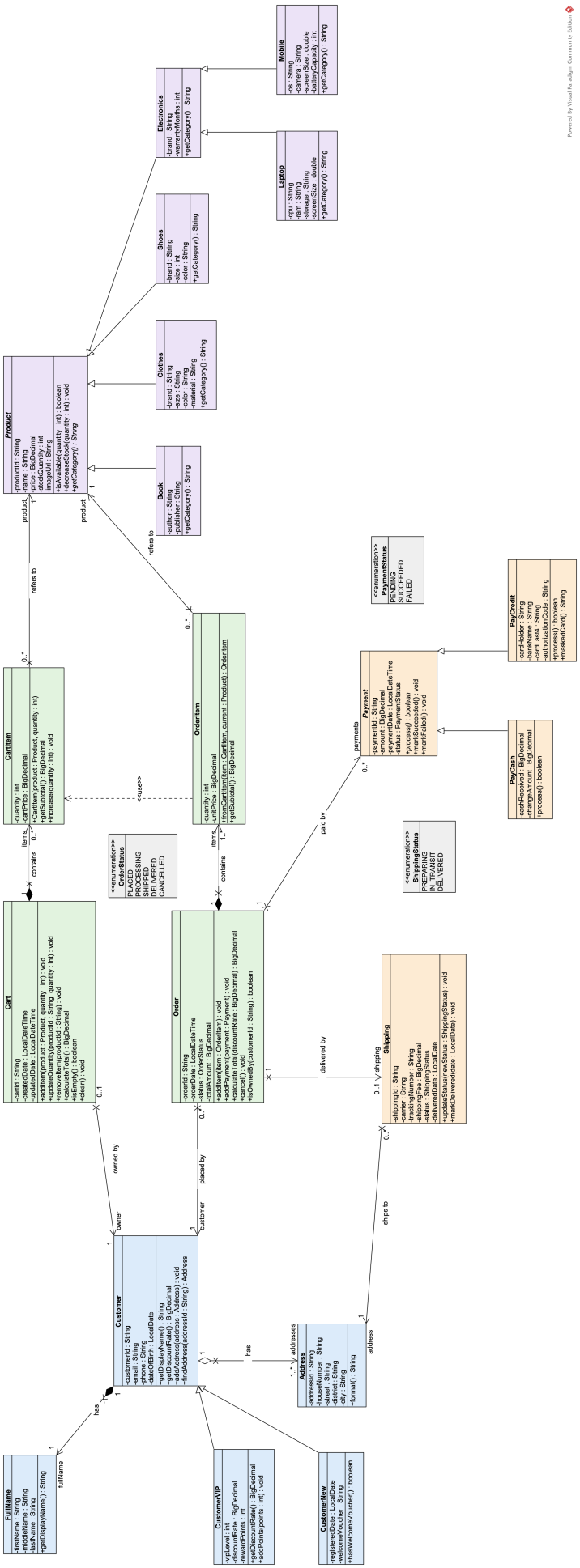
1. DDL executed: 20 tables created (address, book, cart, cart_item, clothes, customer, customer_new, customer_vip, electronics, laptop, mo
2. Sample data: 17 rows inserted into 14 tables without error
3. Negative tests (each statement must be rejected):
   PASS cart_item quantity 0 violates CHECK (quantity > 0): CHECK constraint failed: quantity > 0
   PASS order_item for unknown product P999 violates FOREIGN KEY: FOREIGN KEY constraint failed
   PASS second cart for C001 violates UNIQUE (cart.customer_id), multiplicity 1 : 0..1: UNIQUE constraint failed: cart.customer_id
4. Order O001 subtotal from order_item snapshot prices: 30,300,000
5. Deleting cart CART01 cascades to its cart_item rows: 0 rows left

Result: 20 tables created; 17 rows inserted; 3/3 violations rejected; cascade delete OK
```

Lần chạy này kiểm tra cú pháp và ràng buộc của lược đồ; nó chưa phải phép thử trên máy chủ PostgreSQL thật.

4.3 Class Diagram in Design – Sơ đồ lớp thiết kế

Sơ đồ lớp thiết kế (Hình 4.4) giữ 20 lớp miền của Hình 4.1 và trả lời câu hỏi *mỗi lớp giữ dữ liệu gì, với kiểu nào, và chịu trách nhiệm hành vi nào*. Ba enum trạng thái được thêm vào. Thay đổi chia thành ba phần: thuộc tính, phương thức và quan hệ.



Hình 4.4: Sơ đồ lớp thiết kế: thuộc tính có kiểu và phạm vi truy cập, phương thức, quan hệ có hướng và tên vai trò.

4.3.1 Update attributes – Cập nhật thuộc tính

Bảng 4.8: Thuộc tính ở mức phân tích và mức thiết kế

Phân tích (4.1)	Thiết kế (4.3)	Lý do
`float` cho tiền: `price`, `amount`, `totalAmount`, `unitPrice`, `cartPrice`, `shippingFee`, `discountRate`, `cashReceived`, `changeAmount`	`BigDecimal`	Tiền cần số thập phân chính xác; `float` gây sai số làm tròn khi cộng nhiều dòng hàng.
`Date`: `dateOfBirth`, `registeredDate`, `deliveredDate`	`LocalDate`	Chỉ cần ngày, không cần giờ.
`Date`: `createdDate`, `updatedDate`, `orderDate`, `paymentDate`	`LocalDateTime`	Cần thời điểm chính xác để sắp xếp và đối soát.
`status: String` của `Order`, `Payment`, `Shipping`	Enum `OrderStatus`, `PaymentStatus`, `ShippingStatus`	Giới hạn tập giá trị hợp lệ: `PLACED`, `PROCESSING`, `SHIPPED`, `DELIVERED`, `CANCELLED`; `PENDING`, `SUCCEEDED`, `FAILED`; `PREPARING`, `IN_TRANSIT`, `DELIVERED`. Trình biên dịch phát hiện giá trị sai.
`screenSize: float`	`double`	Kiểu số thực mặc định của Java.
`Address` không có định danh	`addressId: String`	Khi checkout, khách chọn địa chỉ giao bằng mã; `Customer.findAddress(addressId)` cần định danh này.
Phạm vi truy cập chưa xác định	Mọi thuộc tính `private` (dấu −)	Đóng gói: dữ liệu chỉ được đổi qua phương thức của lớp, nơi kiểm tra quy tắc nghiệp vụ.

4.3.2 Update methods – Cập nhật phương thức

Mỗi lớp miền được giao những hành vi gắn với dữ liệu của nó. Bảng 4.9 liệt kê các phương thức trên Hình 4.4. Getter và setter không vẽ trên sơ đồ để sơ đồ gọn; chúng được sinh tự động ở mục 4.5.

Bảng 4.9: Phương thức của các lớp miền

Lớp	Phương thức	Trách nhiệm
`FullName`	`+getDisplayName(): String`	Ghép họ, tên đệm, tên theo thứ tự tiếng Việt, bỏ phần trống.
`Address`	`+format(): String`	Trả chuỗi địa chỉ để in trên đơn và phiếu giao.

Lớp	Phương thức	Trách nhiệm

`Customer`	`+getDisplayName(): String`
	`+getDiscountRate(): BigDecimal`
	`+addAddress(address: Address): void`
	`+findAddress(addressId: String): Address`
		Họ tên hiển thị; tỷ lệ chiết khấu mặc định bằng 0; thêm địa chỉ; trả địa chỉ thuộc khách hoặc báo lỗi nếu địa chỉ không thuộc khách.

`CustomerVIP`	`+getDiscountRate(): BigDecimal` `+addPoints(points: int): void`	Ghi đè để trả `discountRate` của khách VIP; cộng điểm, từ chối số âm.
`CustomerNew`	`+hasWelcomeVoucher(): boolean`	Cho biết khách mới còn voucher chào mừng.

`Product`	`+isAvailable(quantity: int): boolean`
	`+decreaseStock(quantity: int): void`
	`+getCategory(): String {abstract}`
		Kiểm tra đủ tồn kho; giảm tồn và từ chối khi không đủ; trả nhóm hàng, do từng lớp con cài đặt.

`Book`, `Clothes`, `Shoes`, `Electronics`, `Laptop`, `Mobile`	`+getCategory(): String`	Trả `BOOK`, `CLOTHES`, `SHOES`, `ELECTRONICS`, `LAPTOP`, `MOBILE`, khớp giá trị của cột `product.product_type`.

`Cart`	`+addItem(product: Product, quantity: int): void`
	`+updateQuantity(productId: String, quantity: int): void`
	`+removeItem(productId: String): void`
	`+calculateTotal(): BigDecimal`
	`+isEmpty(): boolean`
	`+clear(): void`
		Thêm hàng (cộng dồn nếu sản phẩm đã có trong giỏ); đổi số lượng (số lượng nhỏ hơn hoặc bằng 0 thì xóa dòng); xóa dòng; tính tổng tiền giỏ; kiểm tra giỏ rỗng; làm rỗng giỏ sau checkout.

`CartItem`	`+CartItem(product: Product, quantity: int)`
	`+getSubtotal(): BigDecimal`
	`+increase(quantity: int): void`
		Tạo dòng giỏ và lưu `cartPrice` bằng giá hiện tại; tính thành tiền; cộng thêm số lượng.

`Order`	`+addItem(item: OrderItem): void`
	`+addPayment(payment: Payment): void`
	`+calculateTotal(discountRate: BigDecimal): BigDecimal`
	`+cancel(): void`
	`+isOwnedBy(customerId: String): boolean`
		Thêm dòng đơn; thêm lần thanh toán; tính tổng bằng tổng dòng trừ chiết khấu cộng phí giao, không âm; chỉ hủy được đơn `PLACED` hoặc `PROCESSING`; kiểm tra đơn có thuộc khách hay không.

`OrderItem`	`+fromCartItem(item: CartItem, current: Product): OrderItem {static}` `+getSubtotal(): BigDecimal`	Tạo dòng đơn từ dòng giỏ, chốt `unitPrice` bằng giá hiện hành của sản phẩm; tính thành tiền.
`Shipping`	`+updateStatus(newStatus: ShippingStatus): void` `+markDelivered(date: LocalDate): void`	Cập nhật trạng thái giao; ghi nhận đã giao và ngày giao.

Lớp	Phương thức	Trách nhiệm

`Payment`	`+process(): boolean {abstract}`
	`+markSucceeded(): void`
	`+markFailed(): void`
		Xử lý thanh toán, do lớp con cài đặt; đặt trạng thái thành công (kèm thời điểm) hoặc thất bại.

`PayCash`	`+process(): boolean`	Đủ tiền thì tính tiền thừa và thành công; thiếu tiền thì thất bại.
`PayCredit`	`+process(): boolean` `+maskedCard(): String`	Thành công khi đã có `authorizationCode` từ cổng thanh toán; hiển thị thẻ dạng che số.

Cách giao phương thức theo nguyên tắc *domain responsibility* của slide A05: hành vi gắn với trạng thái của một đối tượng thì đặt trong lớp đó. `Cart.addItem` và `Order.calculateTotal` nằm trong lớp miền, còn lớp service ở mục 4.4 chỉ điều phối quy trình giữa nhiều đối tượng và kho dữ liệu. Ba chỗ dùng đa hình:

- `Payment.process()` là phương thức trừu tượng; `PayCash` và `PayCredit` cài đặt theo cách riêng, nên service chỉ gọi `payment.process()` mà không cần biết hình thức thanh toán.
- `Customer.getDiscountRate()` trả 0, còn `CustomerVIP` ghi đè để trả tỷ lệ chiết khấu của mình. `Order.calculateTotal(customer.getDiscountRate())` vì thế đúng với mọi hạng khách.
- `Product.getCategory()` là trừu tượng; mỗi lớp con trả nhóm hàng của mình.

4.3.3 Update relationships – Cập nhật và giải thích quan hệ

Ở mức phân tích, association chỉ nói hai lớp có liên hệ. Ở mức thiết kế, mỗi association được quyết định thêm *chiều điều hướng* (lớp nào giữ tham chiếu tới lớp nào) và *tên vai trò* (tên field khi cài đặt). Bảng 4.10 tóm tắt.

Bảng 4.10: Quan hệ ở mức thiết kế

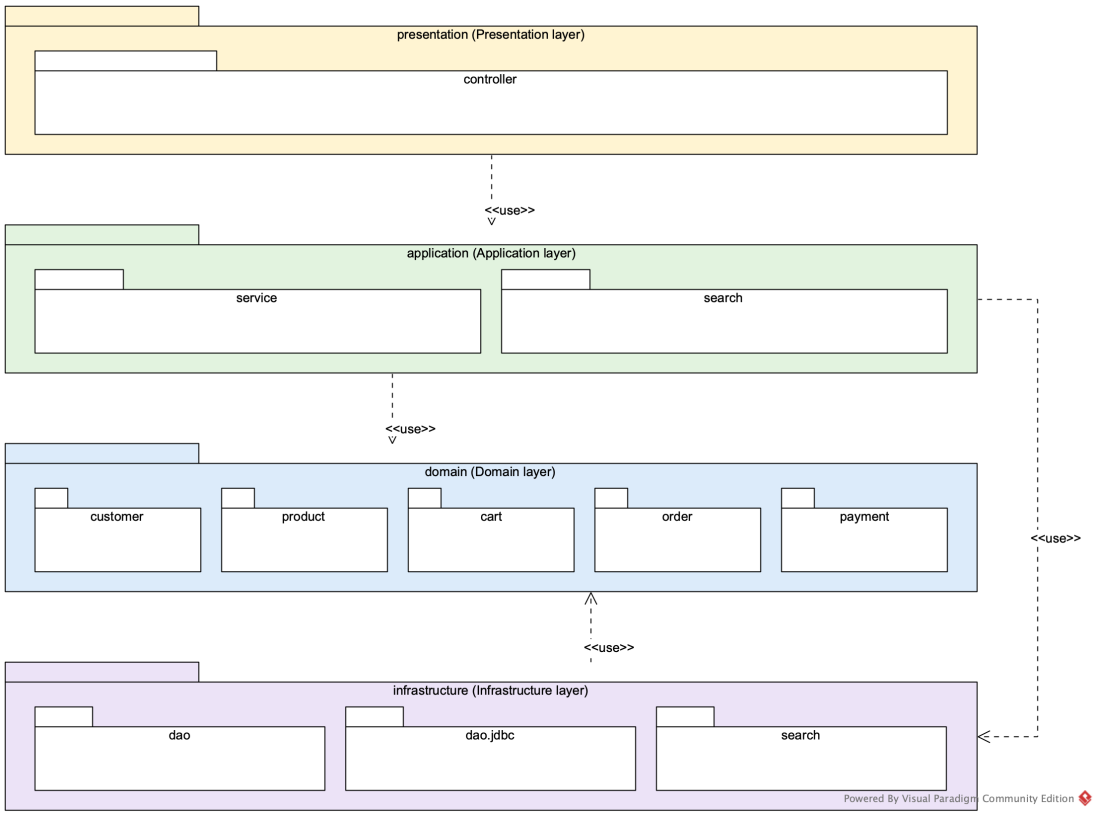
Quan hệ	Hướng và vai trò	Bội số	Giải thích
`Customer` – `FullName` (composition)	`Customer` → `fullName`	1 – 1	Khách giữ đối tượng họ tên của mình.
`Customer` – `Address` (aggregation)	`Customer` → `addresses`	1 – 1..*	Khách giữ danh sách địa chỉ để chọn nơi giao.
`Cart` – `Customer` (owned by)	`Cart` → `owner`	0..1 – 1	Đảo chiều so với *owns*: giỏ biết chủ sở hữu để kiểm tra quyền; khách không giữ tham chiếu tới giỏ, giỏ được tìm bằng `CartDAO.findByCustomerId`.
`Cart` – `CartItem` (composition)	`Cart` → `items`	1 – 0..*	Giỏ quản lý các dòng của nó.
`CartItem` – `Product`	`CartItem` → `product`	0..* – 1	Dòng giỏ biết sản phẩm; sản phẩm không cần biết nó nằm trong giỏ nào.
`Order` – `Customer` (placed by)	`Order` → `customer`	0..* – 1	Đơn biết người đặt (dùng cho `isOwnedBy`); danh sách đơn của khách được đọc qua `OrderDAO.findByCustomer` thay vì giữ trong `Customer`.
`Order` – `OrderItem` (composition)	`Order` → `items`	1 – 1..*	Đơn quản lý các dòng của nó.

Quan hệ	Hướng và vai trò	Bội số	Giải thích
`OrderItem` – `Product`	`OrderItem` → `product`	0..* – 1	Dòng đơn biết sản phẩm đã mua.
`Order` – `Shipping` (delivered by)	`Order` → `shipping`	1 – 0..1	Đơn giữ thông tin giao hàng.
`Shipping` – `Address` (ships to)	`Shipping` → `address`	0..* – 1	Lần giao biết địa chỉ đích.
`Order` – `Payment` (paid by)	`Order` → `payments`	1 – 0..*	Đơn giữ các lần thanh toán.
Kế thừa	`extends`		Giữ nguyên sáu nhánh kế thừa; `Product` và `Payment` là lớp trừu tượng.
`OrderItem` `--→` `CartItem`	«use»		`OrderItem.fromCartItem` đọc dòng giỏ để tạo dòng đơn.

Điều hướng một chiều giảm phụ thuộc vòng: nếu cả `Customer` và `Order` giữ tham chiếu tới nhau, mỗi lần đọc khách sẽ kéo theo toàn bộ đơn. Phụ thuộc «create» *checkout* `Cart` `--→` `Order` của mức phân tích không còn nằm giữa hai lớp miền. Ở mức thiết kế, việc tạo đơn từ giỏ được tách thành hai phần: `OrderItem.fromCartItem` chuyển từng dòng (phụ thuộc «use» `OrderItem` `--→` `CartItem`), còn `CheckoutService` điều phối toàn bộ quy trình và tạo `Order` (phụ thuộc «create» ở Hình 4.6).

4.4 Layer + Class + Relationship – Kiến trúc phân tầng

4.4.1 Layers – Bốn tầng và quy tắc phụ thuộc



Hình 4.5: Bốn layer (package) của hệ thống và phụ thuộc giữa chúng.

Theo slide A05, hệ thống được chia thành bốn tầng Presentation, Application, Domain và Infrastructure (Hình 4.5). Mũi tên nét đứt là phụ thuộc: Presentation dùng Application; Application dùng Domain và Infrastructure; Infrastructure dùng Domain để đọc và ghi đối tượng miền. Không có mũi tên từ Presentation xuống Infrastructure, nghĩa là giao diện không truy cập cơ sở dữ liệu trực tiếp. Domain không phụ thuộc tầng nào, nên quy tắc nghiệp vụ không bị buộc vào framework hay cơ sở dữ liệu. Bên trong mỗi layer là các package con trùng với package Java ở mục 4.5: `controller`; `service`, `search`; `customer`, `product`, `cart`, `order`, `payment`; `dao`, `dao.jdbc`, `search`.

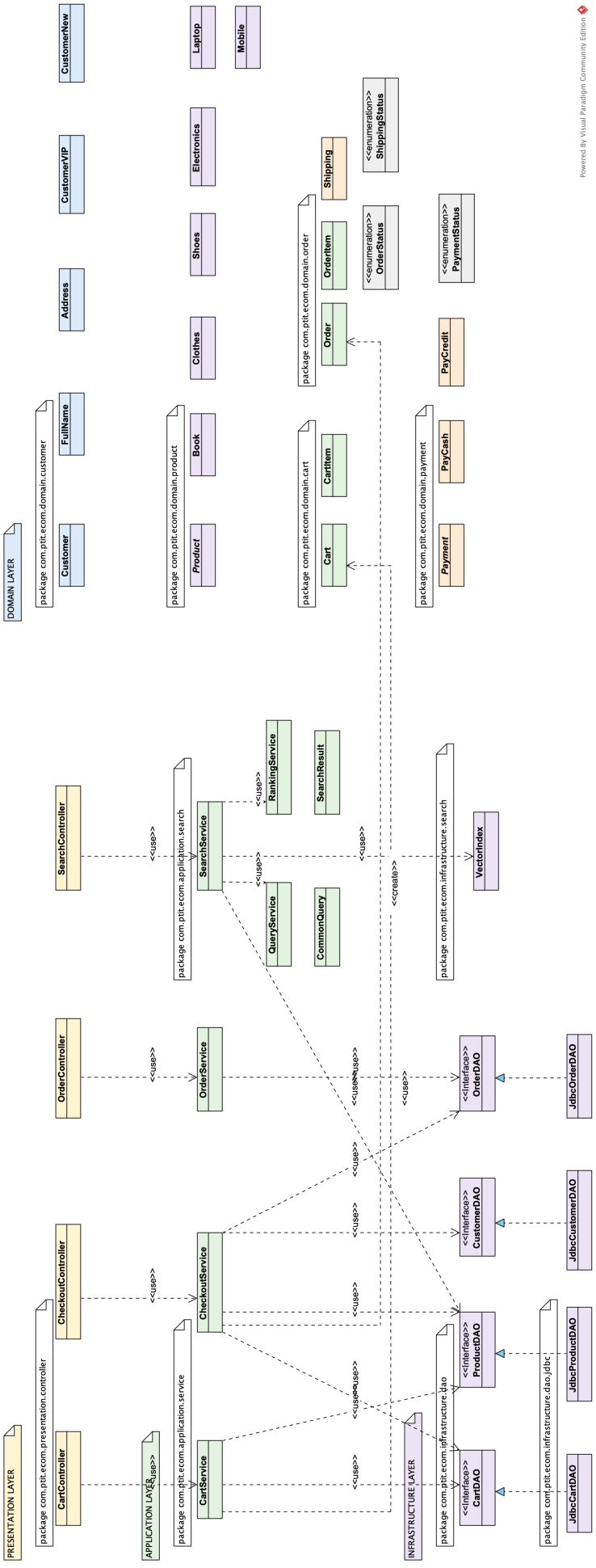
Bảng 4.11: Trách nhiệm của từng tầng

Tầng	Package Java	Lớp	Trách nhiệm	Không được làm
Presentation	`com.ptit.ecom.presentation.controller`	`CartController`, `CheckoutController`, `OrderController`, `SearchController`	Nhận yêu cầu, lấy `customerId` từ phiên đăng nhập, gọi service, trả kết quả	Truy cập CSDL; chứa quy tắc nghiệp vụ

Tầng	Package Java	Lớp	Trách nhiệm	Không được làm
Application	com.ptit.ecom.application.service, com.ptit.ecom.application.search	CartService, CheckoutService, OrderService; SearchService, QueryService, RankingService, CommonQuery, SearchResult	Điều phối use case, xác định ranh giới transaction, gọi DAO	Viết SQL; giữ trạng thái giao diện
Domain	com.ptit.ecom.domain.customer, .product, .cart, .order, .payment	20 lớp miền của mục 4.3 và ba enum trạng thái	Dữ liệu và quy tắc nghiệp vụ (addItem, calculateTotal, cancel, process)	Phụ thuộc framework, CSDL hay giao diện
Infrastructure	com.ptit.ecom.infrastructure.dao, .dao.jdbc, .search	CustomerDAO, ProductDAO, CartDAO, OrderDAO; JdbcCustomerDAO, JdbcProductDAO, JdbcCartDAO, JdbcOrderDAO; VectorIndex	Lưu trữ, câu lệnh SQL, adapter tìm ảnh	Quyết định giá, quyền hay trạng thái đơn

Package không đồng nghĩa với layer (slide A05): layer là trách nhiệm kiến trúc, package là cách tổ chức mã. Ở đây mỗi layer là một package gốc, bên trong chia tiếp theo chức năng (service và search) hoặc theo nhóm miền (cart, order, product, customer, payment). Controller trả về đối tượng miền (Cart, Order) cho phía hiển thị, tức Presentation có đọc kiểu của Domain; bản hoàn chỉnh có thể thay bằng DTO nếu cần ẩn bớt dữ liệu.

4.4.2 Classes and relationships – Lớp và quan hệ trong từng tầng



Hình 4.6: Lớp trong từng tầng và quan hệ *use*, *create*, *realization* giữa chúng.

Hình 4.6 xếp 44 lớp theo bốn tầng: Presentation ở trên, Application ở giữa, Infrastructure ở dưới và Domain bên phải. Mỗi nhãn ghi tên tầng hoặc package Java của nhóm lớp ngay dưới nó; trong project VP, mỗi lớp nằm đúng package con của Hình 4.5. Lớp được vẽ thu gọn (chỉ tên); thuộc tính và phương thức đầy đủ nằm ở Hình 4.4. Có ba loại quan hệ.

Phụ thuộc «use» giữa các tầng. Mỗi controller dùng đúng một service: `CartController` → `CartService`, `CheckoutController` → `CheckoutService`, `OrderController` → `OrderService`, `SearchController` → `SearchService`. Service dùng các interface DAO: `CartService` dùng `CartDAO` và `ProductDAO`; `CheckoutService` dùng `CartDAO`, `OrderDAO`, `ProductDAO`, `CustomerDAO`; `OrderService` dùng `OrderDAO`; `SearchService` dùng `QueryService`, `RankingService`, `ProductDAO` và `VectorIndex`. `CartService` còn dùng `Cart` để gọi hành vi miền. Mỗi phụ thuộc được cài đặt bằng một field nhận qua constructor, ví dụ `CartService(cartDAO: CartDAO, productDAO: ProductDAO)`.

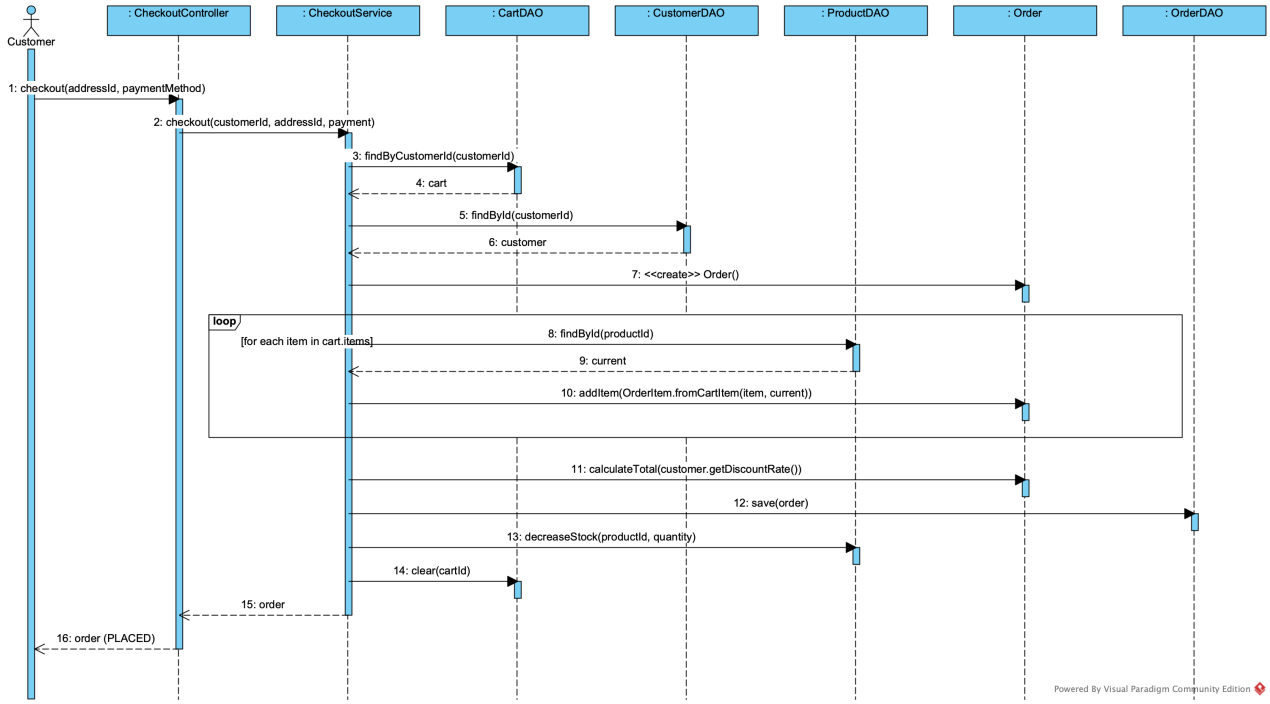
Phụ thuộc «create». `CheckoutService` `--→` `Order`: service tạo đơn mới. Đây là bản thiết kế của phép biến đổi checkout ở mức phân tích.

Realization. `JdbcCustomerDAO`, `JdbcProductDAO`, `JdbcCartDAO` và `JdbcOrderDAO` cài đặt bốn interface tương ứng. Service chỉ phụ thuộc interface, không phụ thuộc lớp JDBC (nguyên tắc dependency inversion). Nhờ vậy có thể đổi hệ quản trị CSDL, hoặc thay bằng DAO trong bộ nhớ khi kiểm thử service, mà không sửa tầng Application.

4.4.3 Explain – Checkout đi qua các tầng

Phương thức `CheckoutService.checkout` là use case trung tâm của slide A05. Trình tự dưới đây bám theo thân phương thức trong đặc tả thiết kế.

1. Khách bấm *Đặt hàng*. `CheckoutController.checkout(customerId, addressId, paymentMethod)` nhận yêu cầu; `customerId` lấy từ phiên đăng nhập.
2. Controller tạo `PayCash` hoặc `PayCredit` theo `paymentMethod`, rồi gọi `CheckoutService.checkout(customerId, addressId, payment)`.
3. Service đọc giỏ bằng `CartDAO.findByCustomerId(customerId)`; giỏ không có hoặc rỗng thì từ chối.
4. Service đọc khách bằng `CustomerDAO.findById(customerId)`.
5. Service tạo `Order` mới với trạng thái `PLACED` và thời điểm đặt hiện tại.
6. Với mỗi `CartItem`, service đọc lại sản phẩm bằng `ProductDAO.findById`, kiểm tra `isAvailable(quantity)`, rồi thêm `OrderItem.fromCartItem(item, current)`. Giá được chốt theo giá hiện hành của catalog, không lấy giá trong giỏ.
7. Service tạo `Shipping` với địa chỉ `customer.findAddress(addressId)` (địa chỉ phải thuộc khách), trạng thái `PREPARING`, rồi gắn vào đơn.
8. `order.calculateTotal(customer.getDiscountRate())` tính tổng tiền theo hạng khách.
9. Service gán số tiền cho `payment`, đặt trạng thái `PENDING` và gọi `order.addPayment(payment)`.
10. `OrderDAO.save(order)`, `ProductDAO.decreaseStock` cho từng dòng và `CartDAO.clear(cartId)` chạy trong một transaction; service trả `Order` cho controller.



Hình 4.7: Sequence diagram thiết kế: checkout đi qua Controller, Service, Domain và DAO.

Hình 4.7 vẽ lại trình tự trên bằng sequence diagram thiết kế trong Visual Paradigm. Mỗi lifeline có kiểu là một lớp của Hình 4.6, nên mỗi thông điệp là một phương thức đã khai báo ở mục 4.3 hoặc 4.4: thông điệp 2 là `CheckoutService.checkout`, các thông điệp 3, 5, 8, 12, 13, 14 đi xuống interface DAO, còn 7, 10, 11 gọi hành vi của lớp miền `Order`. Khung *loop* lặp bước đọc lại sản phẩm và tạo `OrderItem` cho từng dòng giỏ. Đường trả về nét đứt mang kết quả ngược lên tầng trên; không có thông điệp nào đi từ Controller thẳng xuống DAO.

Vì sao dùng `checkout(customerId, addressId, payment)`. Slide A05 gợi ý `CheckoutService.checkout(cartID): Order`. Thiết kế này dùng `customerId` lấy từ phiên đăng nhập. Mỗi khách có tối đa một giỏ (Customer – Cart là 1 – 0..1), nên `customerId` vừa xác định được giỏ, vừa bảo đảm khách chỉ checkout giỏ của chính mình, thay vì tin một `cartId` do trình duyệt gửi lên. `addressId` chọn địa chỉ giao trong danh sách địa chỉ của khách; `payment` là hình thức thanh toán khách đã chọn.

4.4.4 Explain – Gói search

Gói `com.ptit.ecom.application.search` đưa phân hệ tìm kiếm đa phương thức của prototype A6 vào kiến trúc chung. `SearchController.search(mode, text, image)` chọn một trong ba đường:

- **Văn bản:** `SearchService.searchText` gọi `QueryService.buildTextQuery` để chuẩn hóa truy vấn thành `CommonQuery`, lấy ứng viên bằng `ProductDAO.searchByKeyword`, chấm điểm khớp tên, rồi gọi `RankingService.rank`.
- **Giọng nói:** `searchVoice` nhận transcript và đi chung đường văn bản, đúng như cách prototype A6 xử lý transcript.
- **Ảnh:** `searchImage` gọi `VectorIndex.embed` để lấy vector đặc trưng, lấy các sản phẩm gần nhất bằng `VectorIndex.nearest`, đọc lại sản phẩm từ `ProductDAO`, rồi xếp hạng.

Bước lấy ứng viên (retrieval) và bước xếp hạng (ranking) được tách thành hai lớp. `RankingService` loại sản phẩm hết hàng, sắp xếp theo điểm và lấy năm kết quả đầu, nên quy tắc tồn kho vẫn do dữ liệu catalog quyết định. `VectorIndex` là adapter tới engine đặc trưng ảnh (Apple Vision trong prototype

A6); vector được lưu ở bảng `product_feature` kèm `model_version`.

4.5 Generate code from 4.4 – Sinh mã nguồn

4.5.1 Generate code – Cách sinh mã từ mô hình 4.4

Mã Java được sinh bằng script `tools/gen_java.py`. Script đọc cùng tệp đặc tả `tools/model.py` đã dùng để dựng các sơ đồ 4.3 và 4.4 trong Visual Paradigm, nên package, lớp, field và chữ ký phương thức trùng với sơ đồ. Bài không dùng nút Generate Code của Visual Paradigm; nếu phiên bản VP có chức năng sinh mã (Tools > Code), có thể sinh lại từ project để đối chiếu. Kết quả gồm 44 tệp Java (1719 dòng) trong 11 package, đặt tại `code/src`.

Bảng 4.12: Quy tắc ánh xạ từ mô hình UML sang Java

Phần tử UML (mục 4.3, 4.4)	Mã Java	Ví dụ
Package của layer	Package `com.ptit.ecom.<layer>.<nhóm>`	`com.ptit.ecom.domain.cart`
Lớp / lớp trừu tượng	`public class` / `public abstract class`	`Product`, `Payment` là abstract
Lớp «interface»	`public interface`	`OrderDAO`
Enum	`public enum`	`OrderStatus`
Thuộc tính `-name: Type`	Field `private Type name;`	`private BigDecimal unitPrice;`
Đầu association có vai trò, bội số 1 hoặc 0..1	Field kiểu lớp đích	`private Customer owner;`
Đầu association bội số 0..* hoặc 1..*	Field `List` khởi tạo rỗng	`private List<CartItem> items = new ArrayList<>();`
Generalization	`extends`	`CustomerVIP extends Customer`
Realization	`implements`	`JdbcOrderDAO implements OrderDAO`
Phương thức	Phương thức `public`/`private`, cùng chữ ký; `{abstract}` và `{static}` giữ nguyên	`public static OrderItem fromCartItem(...)`
Phương thức trùng tên lớp	Constructor	`CartItem(Product, int)`
Phương thức chưa có thân trong đặc tả	Thân tạm `throw new UnsupportedOperationException("TODO: ...")`	`JdbcOrderDAO.findByCustomer`
Field (không vẽ accessor trên sơ đồ)	Getter và setter sinh tự động	`getItems()`, `setStatus(...)`

4.5.2 Explain packages – Giải thích các package

Cây thư mục mã nguồn do script sinh ra:

```
com/ptit/ecom/
├── application/
│   ├── search/
```

```
│   │   ├── CommonQuery.java
│   │   ├── QueryService.java
│   │   ├── RankingService.java
│   │   ├── SearchResult.java
│   │   └── SearchService.java
│   └── service/
│       ├── CartService.java
│       ├── CheckoutService.java
│       └── OrderService.java
├── domain/
│   ├── cart/
│   │   ├── Cart.java
│   │   └── CartItem.java
│   ├── customer/
│   │   ├── Address.java
│   │   ├── Customer.java
│   │   ├── CustomerNew.java
│   │   ├── CustomerVIP.java
│   │   └── FullName.java
│   ├── order/
│   │   ├── Order.java
│   │   ├── OrderItem.java
│   │   ├── OrderStatus.java
│   │   ├── Shipping.java
│   │   └── ShippingStatus.java
│   ├── payment/
│   │   ├── PayCash.java
│   │   ├── PayCredit.java
│   │   ├── Payment.java
│   │   └── PaymentStatus.java
│   └── product/
│       ├── Book.java
│       ├── Clothes.java
│       ├── Electronics.java
│       ├── Laptop.java
│       ├── Mobile.java
│       ├── Product.java
│       └── Shoes.java
├── infrastructure/
│   ├── dao/
│   │   ├── jdbc/
│   │   │   ├── JdbcCartDAO.java
│   │   │   ├── JdbcCustomerDAO.java
│   │   │   ├── JdbcOrderDAO.java
│   │   │   └── JdbcProductDAO.java
│   │   ├── CartDAO.java
│   │   ├── CustomerDAO.java
│   │   ├── OrderDAO.java
│   │   └── ProductDAO.java
│   └── search/
│       └── VectorIndex.java
└── presentation/
    └── controller/
        ├── CartController.java
        ├── CheckoutController.java
        ├── OrderController.java
        └── SearchController.java
```

Bảng 4.13: Nội dung từng package

Package (`com.ptit.ecom.…`)	Tầng	Nội dung
`presentation.controller`	Presentation	Bốn controller nhận yêu cầu của giỏ, checkout, đơn và tìm kiếm.
`application.service`	Application	`CartService`, `CheckoutService`, `OrderService`: điều phối use case giỏ, đặt hàng, tra cứu và hủy đơn.
`application.search`	Application	`SearchService`, `QueryService`, `RankingService` và hai lớp dữ liệu `CommonQuery`, `SearchResult`.

Package (`com.ptit.ecom.…`)	Tầng	Nội dung
`domain.customer`	Domain	`Customer`, `FullName`, `Address`, `CustomerVIP`, `CustomerNew`.
`domain.product`	Domain	`Product` và sáu lớp con.
`domain.cart`	Domain	`Cart`, `CartItem`.
`domain.order`	Domain	`Order`, `OrderItem`, `Shipping` và hai enum `OrderStatus`, `ShippingStatus`.
`domain.payment`	Domain	`Payment`, `PayCash`, `PayCredit`, enum `PaymentStatus`.
`infrastructure.dao`	Infrastructure	Bốn interface DAO mà tầng Application phụ thuộc.
`infrastructure.dao.jdbc`	Infrastructure	Bốn lớp JDBC cài đặt các interface trên.
`infrastructure.search`	Infrastructure	`VectorIndex`, adapter tới engine đặc trưng ảnh.

Hướng `import` giữa các package đi theo Hình 4.5: `presentation` import `application`; `application` import `domain` và `infrastructure.dao`; `infrastructure` import `domain`; các package `domain` chỉ import lẫn nhau và thư viện chuẩn của Java.

4.5.3 Class code – Giải thích mã của các lớp

Các đoạn dưới đây được trích từ mã đã sinh; phần accessor và các thành phần khác được lược bớt.

Lớp miền `Cart`.

```
public class Cart {
    private String cartId;
    private LocalDateTime createdDate;
    private LocalDateTime updatedDate;
    private Customer owner;

    public void addItem(Product product, int quantity) {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be greater than 0");
        for (CartItem item : items) {
            if (item.getProduct().getProductId().equals(product.getProductId())) {
                item.increase(quantity);
                updatedDate = LocalDateTime.now();
                return;
            }
        }
        items.add(new CartItem(product, quantity));
        updatedDate = LocalDateTime.now();
    }

    public BigDecimal calculateTotal() {
        BigDecimal total = BigDecimal.ZERO;
        for (CartItem item : items) total = total.add(item.getSubtotal());
        return total;
    }
    // ... other members and generated accessors omitted
}
```

Field `owner` và `items` được sinh từ hai đầu association `Cart` → `Customer` và composition `Cart` → `CartItem`. `addItem` cộng dồn số lượng nếu sản phẩm đã có trong giỏ, nên mỗi sản phẩm chỉ có một dòng giỏ; điều này khớp với khóa chính ghép `(cart_id, product_id)` của bảng `cart_item`. `calculateTotal` cộng tiền bằng `BigDecimal`.

Phương thức tĩnh `OrderItem.fromCartItem`.

```
public class OrderItem {
```

```
    private int quantity;
    private BigDecimal unitPrice;
    private Product product;

    public static OrderItem fromCartItem(CartItem item, Product current) {
        OrderItem orderItem = new OrderItem();
        orderItem.product = current;
        orderItem.quantity = item.getQuantity();
        orderItem.unitPrice = current.getPrice();
        return orderItem;
    }

    public BigDecimal getSubtotal() {
        return unitPrice.multiply(BigDecimal.valueOf(quantity));
    }
    // ... other members and generated accessors omitted
}
```

Dòng đơn lấy số lượng từ dòng giỏ nhưng lấy giá từ đối tượng `current` vừa đọc lại từ catalog. `unitPrice` vì thế là giá chốt tại thời điểm đặt, đúng ý nghĩa đã phân tích ở mục 4.1.

Service `CheckoutService`.

```
public class CheckoutService {
    private CartDAO cartDAO;
    private OrderDAO orderDAO;
    private ProductDAO productDAO;
    private CustomerDAO customerDAO;
    // ... constructor injecting the four DAOs omitted

    public Order checkout(String customerId, String addressId, Payment payment) {
        Cart cart = cartDAO.findByCustomerId(customerId);
        if (cart == null || cart.isEmpty()) throw new IllegalStateException("Cart is empty");
        Customer customer = customerDAO.findById(customerId);

        Order order = new Order();
        order.setCustomer(customer);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus(OrderStatus.PLACED);
        for (CartItem item : cart.getItems()) {
            // Price and stock are read again from the catalogue, not trusted from the cart.
            Product current = productDAO.findById(item.getProduct().getProductId());
            if (!current.isAvailable(item.getQuantity()))
                throw new IllegalStateException("Out of stock: " + current.getName());
            order.addItem(OrderItem.fromCartItem(item, current));
        }

        Shipping shipping = new Shipping();
        shipping.setAddress(customer.findAddress(addressId));
        shipping.setShippingFee(BigDecimal.ZERO);
        shipping.setStatus(ShippingStatus.PREPARING);
        order.setShipping(shipping);
        order.calculateTotal(customer.getDiscountRate());

        payment.setAmount(order.getTotalAmount());
        payment.setStatus(PaymentStatus.PENDING);
        order.addPayment(payment);

        // save, decreaseStock and clear must run in one database transaction.
        orderDAO.save(order);
        for (OrderItem item : order.getItems())
            productDAO.decreaseStock(item.getProduct().getProductId(), item.getQuantity());
        cartDAO.clear(cart.getCartId());
        return order;
    }
}
```

Bốn field DAO là các phụ thuộc «use» của Hình 4.6. Thân phương thức là mười bước của mục 4.4.3: service chỉ điều phối, còn việc kiểm tra tồn kho, tạo dòng đơn, tính tổng và quản lý thanh toán được giao cho `Product`, `OrderItem`, `Order` và `Payment`.

DAO `OrderDAO` và `JdbcOrderDAO`.

```java
public interface OrderDAO {
    void save(Order order);
    Order findByIdAndCustomer(String orderId, String customerId);
    List<Order> findByCustomer(String customerId);
    void updateStatus(String orderId, OrderStatus status);
}

public class JdbcOrderDAO implements OrderDAO {
    // ... constructor and other methods omitted

    @Override
    public Order findByIdAndCustomer(String orderId, String customerId) {
        // The customer id comes from the authenticated session, never from the request body.
        String sql = "SELECT order_id, order_date, status, total_amount FROM orders "
                + "WHERE order_id = ? AND customer_id = ?";
        try (Connection connection = dataSource.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, orderId);
            statement.setString(2, customerId);
            try (ResultSet rows = statement.executeQuery()) {
                if (!rows.next()) return null;
                Order order = new Order();
                order.setOrderId(rows.getString("order_id"));
                order.setOrderDate(rows.getTimestamp("order_date").toLocalDateTime());
                order.setStatus(OrderStatus.valueOf(rows.getString("status")));
                order.setTotalAmount(rows.getBigDecimal("total_amount"));
                return order;
            }
        } catch (SQLException e) {
            throw new IllegalStateException("Cannot read order " + orderId, e);
        }
    }
}
```

Interface `OrderDAO` thuộc `infrastructure.dao`; lớp `JdbcOrderDAO` cài đặt nó (realization). `findByIdAndCustomer` dùng `PreparedStatement` với tham số, nên giá trị đầu vào không bị ghép thẳng vào câu SQL. Điều kiện `customer_id` $=$? chỉ trả đơn thuộc khách đang đăng nhập, cài đặt yêu cầu FR-ORDER-02.

Lớp trừu tượng `Payment` và lớp con `PayCash`.

```java
public abstract class Payment {
    private String paymentId;
    private BigDecimal amount;
    private LocalDateTime paymentDate;
    private PaymentStatus status;

    public abstract boolean process();
    // ... other members and generated accessors omitted
}

public class PayCash extends Payment {
    private BigDecimal cashReceived;
    private BigDecimal changeAmount;

    @Override
    public boolean process() {
        if (cashReceived == null || cashReceived.compareTo(getAmount()) < 0) {
            markFailed();
            return false;
        }
        changeAmount = cashReceived.subtract(getAmount());
        markSucceeded();
        return true;
    }
    // ... other members and generated accessors omitted
}
```

`Payment.process()` là phương thức trừu tượng; `PayCash` ghi đè để kiểm tra số tiền nhận và tính tiền thừa. Đây là đa hình đã nêu ở mục 4.3.2.

Gói search.

```
public class SearchService {
    // ... fields, constructor, searchText and searchVoice omitted

    public List<SearchResult> searchImage(byte[] image) {
        CommonQuery query = queryService.buildImageQuery(vectorIndex.embed(image));
        List<SearchResult> candidates = new ArrayList<>();
        for (Map.Entry<String, Double> hit : vectorIndex.nearest(query.getImageVector(), 20).entrySet()) {
            Product product = productDAO.findById(hit.getKey());
            if (product != null) candidates.add(new SearchResult(product, hit.getValue()));
        }
        return rankingService.rank(query, candidates, 5);
    }
}

public class SearchController {
    // ... field and constructor omitted

    public List<SearchResult> search(String mode, String text, byte[] image) {
        if ("voice".equals(mode)) return searchService.searchVoice(text);
        if ("image".equals(mode)) return searchService.searchImage(image);
        return searchService.searchText(text);
    }
}
```

`searchImage` cho thấy phần retrieval (`VectorIndex.nearest` và `ProductDAO.findById`) tách khỏi phần ranking (`RankingService.rank`). `SearchController.search` chỉ chọn đường xử lý theo `mode`.

4.5.4 Biên dịch và kiểm tra nhanh

Mã sinh ra được biên dịch bằng `javac`. Một chương trình kiểm tra nhanh (`code/test/DomainSmokeTest.java`) gọi các phương thức miền có thân cài đặt với dữ liệu mẫu. Kết quả do chương trình in ra:

```
PASS Cart.addItem merges the same product into one line
PASS Cart.calculateTotal = 3 x 150000 + 1 x 30000000 = 30450000
PASS OrderItem.fromCartItem snapshots the current price 160000 (cart price was 150000)
PASS Order.calculateTotal: 30480000 - 10% + 30000 = 27462000 (got 27462000.00)
PASS Order.cancel moves a PLACED order to CANCELLED
PASS Order.cancel rejects a SHIPPED order
PASS PayCash.process with 600000 for 500000 succeeds, change 100000
PASS PayCash.process with 400000 for 500000 fails
PASS Product.decreaseStock(2) leaves 0 laptops
PASS Product.decreaseStock(1) on empty stock is rejected
PASS FullName.getDisplayName = "Vũ Hùng Anh"
ALL CHECKS PASSED (11/11)
```

Các kiểm tra này chỉ bao phủ lớp miền. Các phương thức JDBC chưa được chạy với cơ sở dữ liệu.

4.5.5 Giới hạn của mã sinh ra

- Đây là khung mã từ mô hình thiết kế. Các phương thức chưa có thân trong đặc tả, gồm phần lớn phương thức của `JdbcCustomerDAO`, `JdbcProductDAO`, `JdbcCartDAO`, `JdbcOrderDAO` và hai phương thức của `VectorIndex`, còn ném `UnsupportedOperationException("TODO: ...")`.
- `CheckoutService.checkout` cần một transaction bao ba thao tác lưu đơn, giảm tồn và làm rỗng giỏ; hiện mỗi DAO tự mở kết nối riêng, nên tầng Infrastructure còn phải cung cấp cơ chế transaction.
- Phí giao hàng đang đặt bằng 0; cổng thanh toán thẻ và giao diện người dùng chưa được cài đặt.

Việc tiếp theo cho buổi đánh giá code ngày 05/11: cài đặt các DAO trên lược đồ ở mục 4.2 (hoặc DAO trong bộ nhớ cho demo), thêm transaction cho checkout, viết unit test cho `Cart`, `Order`, `Payment`, và nối `VectorIndex` với engine ảnh của prototype A6.

4.6 Conclusion – Kết luận chương

Chương 4 đi một mạch từ mô hình nghiệp vụ tới mã nguồn. Sơ đồ phân tích 20 lớp (4.1) được chuyển thành data model và 20 bảng có khóa chính, khóa ngoại và ràng buộc (4.2). Sơ đồ lớp thiết kế bổ sung kiểu dữ liệu, phương thức và chiều điều hướng (4.3). Kiến trúc bốn tầng đặt lớp miền cạnh controller, service và DAO (4.4). Cuối cùng, mã Java được sinh từ cùng mô hình đó (4.5). Vì sơ đồ, SQL và mã cùng đọc một tệp đặc tả, tên lớp, tên bảng và chữ ký phương thức nhất quán giữa các mục.

Bảng 4.14: Truy vết từ yêu cầu tới thiết kế, dữ liệu và mã

Yêu cầu	Lớp và phương thức	Bảng, cột	Mục
FR-CART-01: thêm sản phẩm còn bán vào giỏ	`CartController.addItem` → `CartService.addToCart` → `Cart.addItem`, `Product.isAvailable`	`cart`, `cart_item` (`quantity > 0`)	4.3, 4.4, 4.5
FR-ORDER-01: tạo đơn từ giỏ hợp lệ, chốt giá	`CheckoutService.checkout`, `OrderItem.fromCartItem`, `Order.calculateTotal`	`orders`, `order_item.unit_price`	4.3, 4.4.3, 4.5
FR-ORDER-02: khách chỉ xem đơn của mình	`OrderService.getOrder` → `OrderDAO.findByIdAndCustomer`; `Order.isOwnedBy`	`orders.customer_id`	4.4, 4.5
FR-SEARCH-01: tìm theo từ khóa	`SearchService.searchText`, `QueryService.buildTextQuery`, `RankingService.rank`	`product`	4.4.4, 4.5
FR-SEARCH-03: tìm bằng ảnh	`SearchService.searchImage`, `VectorIndex.nearest`	`product_feature`	4.2, 4.4.4
D-01: không lưu số thẻ đầy đủ	`PayCredit.cardLast4`, `authorizationCode`, `maskedCard`	`pay_credit.card_last4`	4.1, 4.2, 4.3

Hạn chế. Mô hình dữ liệu chưa có lịch sử chuyển trạng thái đơn và chưa có tồn kho theo kho hàng. Phí giao hàng chưa được thiết kế. Ràng buộc "đúng một bản ghi con" của cột phân loại và cận dưới 1..* vẫn do tầng nghiệp vụ giữ. Mã ở mục 4.5 là khung có một phần thân cài đặt, chưa phải ứng dụng chạy được.

Kế hoạch. Ngày 29/10 đánh giá phần thiết kế; ngày 05/11 đánh giá code, với các việc nêu ở cuối mục 4.5; ngày 12/11 và 19/11 đánh giá tiểu luận final và demo của từng sinh viên. Khi ghép vào tiểu luận final, chương này thay chương Design của bản V1.

Phụ lục – Tệp nộp kèm và cách tái tạo

Tệp, thư mục (trong `tieuluan/`)	Nội dung
`vp/tieuluan_design.vpp`	Project Visual Paradigm chứa toàn bộ sơ đồ của chương, mở và chỉnh sửa được
`figures/vp/*.png`, `*.svg`	Ảnh sơ đồ xuất từ project
`db/ecom_schema.sql`	DDL PostgreSQL của 20 bảng
`code/src`	Mã Java sinh từ mô hình 4.4
`tools/model.py`	Đặc tả chung của lớp, quan hệ, bảng
`chapter4/design_04.tex`	Mã nguồn LaTeX của chương

Lệnh tái tạo, chạy trong thư mục `tieuluan/`: `tools/build_vp.py` dựng project VP và xuất hình; `tools/gen_sql.py` sinh `db/ecom_schema.sql` và kiểm tra trên SQLite; `tools/gen_java.py` sinh `code/src`, biên dịch và chạy kiểm tra nhanh; lệnh cuối biên dịch chương này.

```
python3 tools/build_vp.py
python3 tools/gen_sql.py
python3 tools/gen_java.py
tectonic -X compile chapter4/design_04.tex --outdir build
```